DAILY ENGINEERING REPORT

ESP32 TFT Display Setup, Touchscreen Integration & Dial Pad

Work completed: 30 October 2026

Project detail	Configuration / status
Main controller	ESP32 DevKit V1
Display	3.5-inch SPI TFT LCD, 480 × 320, ILI9488 driver
Touch controller	XPT2046 resistive touchscreen
Development environment	Arduino IDE, Arduino framework / C++
Display library	TFT_eSPI
Touch library	XPT2046_Touchscreen by Paul Stoffregen
Today's result	Display rendering and touch-controller communication work; final touch alignment remains to be verified.
Next session	1) Display scaling and layout calibration 2) Test display and AS608 fingerprint sensor together

1. Executive Summary

Today's work focused on bringing up the ESP32 with the 3.5-inch ILI9488 TFT display, configuring the TFT_eSPI library, testing the XPT2046 touchscreen, and creating a telephone-style dial-pad interface. The display was successfully initialized and the keypad was drawn on the screen. The touchscreen initially returned invalid readings. The communication issue was resolved by disconnecting the display's SDO/MISO output from GPIO19 while keeping the touchscreen's T_DO/MISO connected to GPIO19.

After touch communication started working, the keypad's touch zones did not consistently match the visible buttons. Raw coordinates were captured at the four corners and an initial mapping was added to the full sketch. This mapping is provisional and must be validated before calling the touch alignment complete.

The next work session is planned around two tasks: (1) display scaling/layout calibration and (2) testing the TFT display and AS608 fingerprint sensor together.

2. Objectives for Today

- Configure the ILI9488 display driver and ESP32 SPI pins.
- Initialize the TFT display and verify that graphics render.
- Initialize the XPT2046 touchscreen and read raw coordinates.
- Diagnose the touch communication failure.
- Build a dial-pad UI with digits 0–9, * and #, plus DEL, CLEAR and CALL controls.
- Collect corner readings and add a first-pass touch coordinate mapping.

3. Devices and Specifications

Device / item	Known specification	Role / note
ESP32 DevKit V1	ESP32 development board; 3.3 V GPIO logic	Runs the interface and communicates with the display and touch controller.
TFT LCD module	3.5-inch SPI TFT; 480 × 320 pixels; ILI9488 controller	Displays keypad and entered number.
Touch controller	XPT2046 resistive touch controller	Reports raw touch coordinates and pressure.
TFT_eSPI	Arduino display library	Drawing, text, rotation and display initialization.
XPT2046_Touchscreen	Arduino touchscreen library by Paul Stoffregen	Touch detection and raw point acquisition.
AS608 fingerprint sensor	Sensor model identified for planned integration; exact module voltage/pin labels should be checked on the hardware	Planned next-session peripheral; not yet tested with the display in this session.
USB cable / jumper wires	Used for programming, power and signals	Connects board and peripherals.

Power note: confirm the exact TFT module's VCC and backlight requirements from its PCB/manual. Do not assume every 3.5-inch module accepts 5 V on all pins. ESP32 GPIOs use 3.3 V logic.

4. Pin Configuration Used

4.1 TFT display

TFT pin label	ESP32 GPIO	Purpose / current state
SCK	GPIO18	SPI clock; shared with touch controller.
SDI / MOSI	GPIO23	SPI data from ESP32; shared with touch controller.
CS	GPIO5	TFT chip-select.
DC / RS	GPIO2	Display data/command control.

RESET	GPIO4	Display reset.
SDO / MISO	Disconnected	Deliberately disconnected to avoid SPI data-line interference with touch controller.
GND	GND	Common ground.
VCC / LED	According to module specification	Power and backlight; verify the exact board requirements.

4.2 Touch controller

Touch pin label	ESP32 GPIO	Purpose / current state
T_CLK	GPIO18	Shared SPI clock.
T_DIN / T_MOSI	GPIO23	Data to touch controller.
T_DO / T_MISO	GPIO19	Touch data output; keep connected.
T_CS	GPIO22	Touch chip-select; separate from TFT CS.
T_IRQ	Not connected	Optional interrupt pin; polling is used in current code.

The display and touch controller share SCK and MOSI, but they must have separate CS pins. The TFT SDO/MISO is disconnected; the touch T_DO/MISO remains on GPIO19. Do not reconnect TFT SDO to GPIO19 while using this shared SPI arrangement unless the module's bus behavior is independently verified.

5. Arduino IDE and Library Setup

The project was compiled in Arduino IDE using an ESP32 board selection (ESP32 Dev Module / the appropriate DevKit V1 entry). The following libraries are required:

- TFT_eSPI — display graphics and ILI9488 support.
- XPT2046_Touchscreen by Paul Stoffregen — touch detection and coordinate readings.

The TFT_eSPI User_Setup.h must select the ILI9488 driver, not ILI9341, and use the ESP32 pin configuration shown in this report. Any ESP8266-specific pin definitions must remain disabled. User_Setup_Select.h should select the intended User_Setup.h configuration without another conflicting setup.

```
#define ILI9488_DRIVER

#define TFT_MISO 19
#define TFT_MOSI 23
#define TFT_SCLK 18
#define TFT_CS   5
#define TFT_DC   2
#define TFT_RST  4

#define TOUCH_CS 22
```

Note: TFT_eSPI's TFT_MISO definition is still a library configuration value, but the physical TFT SDO/MISO wire is intentionally disconnected. This project only writes graphics to the TFT; it does not need TFT readback for the current UI.

6. Work Completed Today

6.1 Display initialization

- Reviewed the TFT_eSPI setup and selected ILI9488 as the active driver.
- Configured the ESP32 pins: SCK 18, MOSI 23, TFT CS 5, DC 2 and RESET 4.
- Initialized the display using tft.init().
- Set landscape orientation with tft.setRotation(1).
- Verified that the display rendered the dial-pad screen.

6.2 Dial-pad interface

A 480 × 320 screen layout was created. The keypad has four rows and three columns: 1–3, 4–6, 7–9, and *, 0, #. Three bottom controls were added: DEL, CLEAR and CALL. The code uses a shared button-drawing function and calculates each keypad button's position from its row and column.

6.3 Touch-controller test

A separate XPT2046 test sketch was used before integrating touch input with the dial pad. The test printed raw X, Y and pressure readings to the Serial Monitor at 115200 baud.

6.4 Touchscreen integration

After correcting the SPI MISO conflict, the touch controller began producing usable touch readings. The touch input was then connected to the dial-pad logic. The code maps raw values to screen coordinates, checks which button rectangle contains the point, and runs the relevant action.

7. Problems Encountered and Solutions

Problem 1 — Driver/configuration mismatch risk

Issue: The TFT_eSPI setup had more than one display-driver definition enabled at one point, including ILI9341 and ILI9488, and ESP8266-specific pin definitions were present in the setup file.

Resolution: ILI9488 was retained as the active display driver, and the setup was configured for the ESP32 pin assignments. The display subsequently initialized and rendered the keypad.

Status: Resolved for the current display.

Problem 2 — Touch readings stuck at X=0, Y=0, Pressure=4095

Observed output (representative):

```
X: 0 Y: 0 Pressure: 4095
X: 0 Y: 0 Pressure: 4095
...
```

Issue: The touchscreen library appeared to detect a touch continuously, but the coordinates were not valid. The TFT SDO/MISO and touch T_DO/MISO were both connected to GPIO19, which can cause contention on the shared MISO line with some ILI9488 modules.

Resolution: Disconnected the TFT display's SDO/MISO wire from GPIO19. Kept the touch controller's T_DO/MISO connected to GPIO19. The user confirmed that touch then worked.

Status: Resolved in the tested wiring configuration.

Problem 3 — Touch points did not match the visible buttons

Issue: The keypad was drawn correctly, but pressing one button could enter another digit—for example, pressing 1 could register 9. The initial code used assumed raw limits of 200–3800 and did not account for the actual axis orientation and ranges.

Resolution attempted: Collected four corner readings and replaced the assumed mapping with a first-pass map() transformation. Screen coordinates are constrained to X=0–479 and Y=0–319.

Status: Initial calibration added; final alignment remains unverified. Do not report the keypad as fully calibrated until each button and edge has been tested.

8. Touch Calibration Readings and Mapping

The user reported starting at the top-right corner and moving clockwise. The readings were:

Corner	Raw X	Raw Y
Top-right	505	3626

Bottom-right	715	849
Bottom-left	3816	266
Top-left	3610	3389

The provisional mapping included in the latest full sketch is:

```
int screenX = map(p.y, 3626, 266, 0, 479);
int screenY = map(p.x, 505, 3816, 0, 319);

screenX = constrain(screenX, 0, 479);
screenY = constrain(screenY, 0, 319);
```

Important limitation: the readings were taken near the corners but vary considerably. They are a starting point, not a mathematically validated calibration. On the next session, test button centers and, if needed, use a calibration screen or fit the coordinate transformation from multiple known points.

9. Current Dial-Pad Code

The following is the full integrated sketch from today. It uses the current pin setup and provisional coordinate mapping. The CALL button only displays/logs the number; it does not place a real telephone call.

```
#include <SPI.h>
#include <TFT_eSPI.h>
#include <XPT2046_Touchscreen.h>

// Pin configuration
#define TFT_CS_PIN 5
#define TOUCH_CS 22

// Screen dimensions
#define SCREEN_WIDTH  480
#define SCREEN_HEIGHT 320

// Button dimensions
#define BUTTON_W 90
#define BUTTON_H 48
#define GAP 10

// Dial pad starting position
#define START_X 95
#define START_Y 65

// Colors
#define BG_COLOR      TFT_BLACK
#define BUTTON_COLOR  0x3186
#define TEXT_COLOR    TFT_WHITE
#define GREEN_COLOR   TFT_GREEN
#define RED_COLOR     TFT_RED
#define BLUE_COLOR    TFT_BLUE

TFT_eSPI tft = TFT_eSPI();
XPT2046_Touchscreen touch(TOUCH_CS);

String phoneNumber = "";

const char* keys[12] = {
  "1", "2", "3",
  "4", "5", "6",
  "7", "8", "9",
  "*", "0", "#"
};

void drawButton(int x, int y, int w, int h,
                const char* label, uint16_t color) {
  tft.fillRoundRect(x, y, w, h, 8, color);
  tft.drawRoundRect(x, y, w, h, 8, TFT_WHITE);

  tft.setTextDatum(MC_DATUM);
  tft.setTextColor(TEXT_COLOR, color);
```

```
  tft.setTextSize(2);
  tft.drawString(label, x + w / 2, y + h / 2);
}

void drawPhoneNumber() {
  tft.fillRect(35, 30, 410, 27, BG_COLOR);

  tft.setTextDatum(MC_DATUM);
  tft.setTextColor(TFT_YELLOW, BG_COLOR);
  tft.setTextSize(2);

  String displayNumber = phoneNumber;

  if (displayNumber.length() > 23) {
    displayNumber =
      displayNumber.substring(displayNumber.length() - 23);
  }

  tft.drawString(displayNumber, 240, 43);
}

void drawDialPad() {
  tft.fillScreen(BG_COLOR);

  tft.setTextDatum(MC_DATUM);
  tft.setTextColor(TFT_CYAN, BG_COLOR);
  tft.setTextSize(2);
  tft.drawString("TOUCH DIAL PAD", 240, 15);

  drawPhoneNumber();

  for (int i = 0; i < 12; i++) {
    int row = i / 3;
    int col = i % 3;

    int x = START_X + col * (BUTTON_W + GAP);
    int y = START_Y + row * (BUTTON_H + GAP);

    drawButton(x, y, BUTTON_W, BUTTON_H, keys[i], BUTTON_COLOR);
  }

  drawButton(25, 280, 125, 30, "DEL", BLUE_COLOR);
  drawButton(177, 280, 125, 30, "CLEAR", RED_COLOR);
  drawButton(330, 280, 125, 30, "CALL", GREEN_COLOR);
}

void handleTouch(int x, int y) {
  // Number buttons
  for (int i = 0; i < 12; i++) {
    int row = i / 3;
    int col = i % 3;

    int bx = START_X + col * (BUTTON_W + GAP);
    int by = START_Y + row * (BUTTON_H + GAP);

    if (x >= bx && x <= bx + BUTTON_W &&
        y >= by && y <= by + BUTTON_H) {
      phoneNumber += keys[i];
      drawPhoneNumber();

      Serial.print("Number: ");
      Serial.println(phoneNumber);
      return;
    }
  }

  // DEL button
  if (x >= 25 && x <= 150 && y >= 280 && y <= 310) {
    if (phoneNumber.length() > 0) {
      phoneNumber.remove(phoneNumber.length() - 1);
      drawPhoneNumber();
    }
    Serial.println("Last digit deleted");
    return;
```

```
  }

  // CLEAR button
  if (x >= 177 && x <= 302 && y >= 280 && y <= 310) {
    phoneNumber = "";
    drawPhoneNumber();
    Serial.println("Number cleared");
    return;
  }

  // CALL button
  if (x >= 330 && x <= 455 && y >= 280 && y <= 310) {
    tft.fillRect(20, 250, 440, 25, BG_COLOR);

    tft.setTextDatum(MC_DATUM);
    tft.setTextColor(GREEN_COLOR, BG_COLOR);
    tft.setTextSize(2);
    tft.drawString("CALL: " + phoneNumber, 240, 250);

    Serial.print("CALL: ");
    Serial.println(phoneNumber);
    return;
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(TFT_CS_PIN, OUTPUT);
  digitalWrite(TFT_CS_PIN, HIGH);

  pinMode(TOUCH_CS, OUTPUT);
  digitalWrite(TOUCH_CS, HIGH);

  // ESP32 SPI: SCK, MISO, MOSI, SS
  SPI.begin(18, 19, 23, -1);

  tft.init();
  tft.setRotation(1);

  touch.begin();
  touch.setRotation(1);

  drawDialPad();
  Serial.println("Touch dial pad ready!");
}

void loop() {
  if (touch.touched()) {
    TS_Point p = touch.getPoint();

    // Provisional calibration from today's corner readings
    int screenX = map(p.y, 3626, 266, 0, 479);
    int screenY = map(p.x, 505, 3816, 0, 319);

    screenX = constrain(screenX, 0, 479);
    screenY = constrain(screenY, 0, 319);

    Serial.print("Screen X: ");
    Serial.print(screenX);
    Serial.print(" | Screen Y: ");
    Serial.println(screenY);

    handleTouch(screenX, screenY);

    // Wait for release to avoid repeated input
    while (touch.touched()) {
      delay(10);
    }

    delay(80);
  }
```

```
  delay(10);
}
```

10. Expected Output and Observations

10.1 Display output

The TFT should display a black background, a cyan “TOUCH DIAL PAD” heading, a yellow number field, 12 number/symbol buttons, and three bottom control buttons: DEL, CLEAR and CALL.

10.2 Serial Monitor output

Set the Serial Monitor to 115200 baud. On startup, the program prints:

```
Touch dial pad ready!
```

When a button is recognized, output is similar to:

```
Screen X: <mapped x> | Screen Y: <mapped y>
Number: 1
Number: 12
Number cleared
Last digit deleted
CALL: 012345
```

These are examples of expected messages, not a claim that every control has already passed final testing.

10.3 Touch calibration output collected

```
X = 505  | Y = 3626
X = 715  | Y = 849
X = 3816 | Y = 266
X = 3610 | Y = 3389
```

11. Known Limitations / Risks

- Touch calibration remains provisional; corner readings should be rechecked and button centers tested.
- The current button geometry places the fourth keypad row near the lower screen area; verify the full keypad and bottom controls fit with the actual rotation and dimensions.
- The code uses Arduino String for the number buffer; this is acceptable for a small demo, but a fixed-size character array may be preferable in a long-running embedded application.
- The CALL button is not connected to a telephone/cellular module.
- The AS608 fingerprint sensor has not yet been tested concurrently with the TFT in this session.
- Confirm module power requirements before connecting the TFT or AS608 sensor; do not assume all module pins tolerate 5 V.

12. Next-Day Plan

Task 1 — Display scaling and layout calibration

Objective: make the visible layout fit the display cleanly and make every touch region match its corresponding button.

- Confirm actual display width/height after tft.setRotation(1), using 480 × 320 as the expected landscape dimensions.
- Review the layout coordinates, button sizes, gaps, title, number field and bottom controls.
- Draw a calibration screen with visible targets at known screen coordinates.
- Record raw X/Y values at the center of each target and at the four corners.
- Calculate axis direction, scale and offset; verify whether the current axis swap is correct.
- Apply the new mapping and test buttons 1–9, *, 0, #, DEL, CLEAR and CALL.
- Test touches near edges and check for duplicate input from one press.

- Save the verified pin map, calibration constants and final sketch as the baseline version.

Task 2 — Test TFT display and AS608 fingerprint sensor together

Objective: verify that the display/touch system and fingerprint sensor can operate together without SPI conflicts, serial conflicts, resets or power problems.

- Inspect the AS608 module labels and confirm its supply-voltage and UART logic requirements from the module documentation.
- Keep the TFT and XPT2046 SPI wiring as currently working.
- Connect the AS608 sensor using a hardware UART on the ESP32. A common option is UART2 with ESP32 RX2 = GPIO16 and TX2 = GPIO17, but verify the board and sensor wiring before connecting.
- Cross the UART data lines: sensor TX to ESP32 RX, and sensor RX to ESP32 TX. Connect common ground. Confirm whether the sensor requires 3.3 V or 5 V supply and whether its UART inputs tolerate ESP32 3.3 V logic.
- Install and identify the appropriate fingerprint library (for example, Adafruit Fingerprint Sensor Library if compatible with the exact AS608 module).
- First run a standalone fingerprint test to confirm sensor communication and sensor ID/status.
- Then integrate fingerprint communication with the display program. Avoid using SPI pins 18, 19 and 23 for the UART sensor.
- Show status messages on the TFT, such as “Place finger”, “Fingerprint detected”, “Match found” or “No match”.
- Test display rendering and touch input while the fingerprint sensor is connected.
- Monitor the Serial Monitor for UART errors, ESP32 resets, SPI/touch failures and unstable power.
- Document the final pin diagram, library versions and test results.

Suggested initial AS608 UART wiring (confirm with the actual sensor board before powering):

AS608 signal	ESP32 connection	Notes
Sensor TX	GPIO16 (ESP32 UART RX2)	UART data from sensor to ESP32.
Sensor RX	GPIO17 (ESP32 UART TX2)	UART data from ESP32 to sensor.
GND	GND	Common ground is required.
VCC	According to sensor module specification	Check the exact breakout board voltage requirement before connecting.

Important: GPIO16/17 are a proposed UART2 assignment, not a connection already tested in today’s session. Verify the exact AS608 module pinout and power specification first.

13. Tomorrow's Acceptance Checklist

Check	Acceptance condition	Status
Display scaling	All UI elements fit within 480 × 320 landscape area without clipping.	Pending
Touch mapping	Each number button registers its own label.	Pending
Control buttons	DEL, CLEAR and CALL respond in their intended areas.	Pending
Touch stability	One press produces one action; release permits the next press.	Pending
AS608 standalone	ESP32 can communicate with fingerprint sensor and report status.	Pending
Combined operation	Display, touch and fingerprint sensor work together without interference or resets.	Pending
Documentation	Final pin map, calibration values, code and test results saved.	Pending

14. Conclusion

Today’s session achieved the key display bring-up milestone and restored touchscreen communication. The ILI9488 display renders the dial-pad interface, and the XPT2046 touch controller now returns usable readings after the display SDO/MISO line was disconnected from GPIO19. A first-pass coordinate mapping has been included in the complete sketch, but the final alignment of touch areas and visible buttons remains to be validated.

The next session will focus on display scaling and calibration first, followed by standalone AS608 fingerprint testing and combined integration with the TFT/touch system. The key goal is to preserve the working SPI configuration while adding the fingerprint sensor over a separate UART interface.

Appendix A — Key Pin Summary

Function	GPIO
Shared SPI SCK: TFT + touch	18
Shared SPI MOSI: TFT + touch	23
Touch MISO (T_DO)	19
TFT SDO/MISO physical wire	Disconnected
TFT CS	5
TFT DC	2
TFT RESET	4
Touch CS	22
Proposed AS608 UART2 RX	16
Proposed AS608 UART2 TX	17